RyuGraph Performance Reference

This document captures performance characteristics, threading models, and optimal configuration for RyuGraph (KuzuDB fork) in the Graph OLAP Platform context.

Prerequisites

Read these documents first:

- ryugraph-networkx.reference.md - Core RyuGraph/KuzuDB capabilities
- data-pipeline.reference.md - COPY FROM syntax and data flow

Related Documents

- ryugraph-wrapper.design.md - Wrapper implementation
- system.architecture.design.md - Pod specifications

KuzuDB Architecture Overview

Storage Model

KuzuDB uses a **memory-mapped, disk-backed** storage architecture:

```
┌──────────────────────────────────────────────────────────┐
│                        Application                       │
├──────────────────────────────────────────────────────────┤
│                        Buffer Pool                       │
│                 (configurable size)                      │
│   ┌──────────┐  ┌──────────┐  ┌──────────┐  ┌──────────┐ │
│   │ Page 1   │  │ Page 2   │  │ Page 3   │  │   ...    │ │
│   └──────────┘  └──────────┘  └──────────┘  └──────────┘ │
├──────────────────────────────────────────────────────────┤
│                 Disk Storage (Persistent)                │
```

```
│          + Spill Files (Temporary, auto-cleaned)          │
└──────────────────────────────────────────────────────────┘
```

Key characteristics:

- **Buffer pool**: In-memory cache for frequently accessed pages
- **Disk spilling**: Automatic overflow to disk when buffer pool is full
- **Columnar storage**: Data stored in columns for vectorized processing
- **MVCC**: Multi-version concurrency control for transactions

Concurrency Model

KuzuDB uses a **single-writer, multiple-reader** model:

Operation	Concurrency	Notes
Read queries	Parallel	Multiple connections can read simultaneously
Write transactions	Serialized	Only one write at a time (internal locking)
COPY FROM	Serialized	Each COPY FROM is a write transaction

Important: Even with multiple connections, write operations (including COPY FROM) are serialized internally by KuzuDB's transaction manager.

COPY FROM Pipeline Architecture

Internal Threading Model

From KuzuDB COPY Pipeline documentation:

```
┌──────────────────────────────────────────────────────────────┐
│                     COPY FROM Pipeline                       │
│                                                              │
│   ┌─────────┐     ┌──────────┐     ┌──────────────┐     ┌──────┐   │
```

```
│    │ READER   │──→│ COPY_NODE │──→│ BUILD_INDEX │──→│ SINK │   │
│    │ Thread1 │     └──────────┘     └──────────────┘     └──────┘   │
│    ├──────────┤                                                      │
│    │ READER   │     All READERs share ReaderSharedState              │
│    │ Thread2 │     (coordinates file chunk distribution)            │
│    ├──────────┤                                                      │
│    │ READER   │     Each processes "morsels" (2048 tuples)           │
│    │ Thread3 │     independently                                    │
│    └──────────┘                                                      │
└──────────────────────────────────────────────────────────────────────┘
```

Morsel-driven parallelism: Work is divided into small chunks (morsels) that are dynamically distributed across threads during runtime. Each thread processes its chunk independently, synchronizing only when necessary.

Node vs Relationship Loading

Node COPY Pipeline:

```
READER → COPY_NODE → BUILD_INDEX → COPY_SINK
```

Relationship COPY Pipeline (more complex):

```
Pipeline 1 (child):  READER → INDEX_LOOKUP → REL_SHUFFLE
Pipeline 2 (parent): DATA_CHUNK_SCAN → COPY_REL_COLUMNS → COPY_REL_LISTS →
COPY_SINK
```

The REL_SHUFFLE operator accumulates and partitions relationship data for node-group-specific processing.

Serial vs Non-Serial Mode

Mode	Use Case	Behavior
Serial	Node tables with SERIAL primary key	Sequential reads, one node group at a time
Non-Serial	All other tables	Parallel reads, 2048-tuple chunks

I/O Characteristics

Time Distribution During COPY FROM GCS

```
Time breakdown per operation (approximate):

├── Network I/O (GCS HTTP requests) ──────────────── 70-85%
│   └── Latency: 50-200ms per request
│   └── Waiting for bytes over network
│
├── Parquet Decoding ──────────────────────────── 10-20%
│   └── Decompression (snappy/zstd)
│   └── Column parsing
│   └── SIMD-optimized, fast
│
├── Memory/Buffer Operations ──────────────────── 3-8%
│   └── Buffer pool allocation
│   └── Data structure construction
│
└── Disk I/O (storage write) ──────────────────── 2-5%
    └── SSD/NVMe is fast
    └── Only bottleneck at very high throughput
```

Key insight: COPY FROM GCS is **I/O bound**, not CPU bound. Network latency dominates execution time.

Evidence: Benchmarks

From KuzuDB 0.7.0 Release:

Buffer Pool	Load Time (17B edges)	Relative
420 GB	60 min	1.0x
205 GB	68 min	1.13x
102 GB	70 min	1.17x

Analysis: 4x less memory = only 17% slower. This confirms I/O-bound behavior.

From DuckDB GCS Performance Issue:

Method	Time
Sequential HTTP	100s
8 parallel threads	15-17s
Speedup	**6x**

The bottleneck is **request-level concurrency**, not bandwidth or CPU.

Threading Configuration

Why More Threads Than CPUs

For I/O-bound workloads, optimal thread count follows:

```
optimal_threads = CPUs × (1 + wait_time / compute_time)
```

For GCS Parquet loading:

- Wait time (network): ~100ms average
- Compute time (decode): ~10ms average
- Ratio: **10:1**
- Optimal: `CPUs × 4–10`

CPU Utilization Patterns

With threads = CPUs (suboptimal):

```
Thread 1: [WAIT GCS ~~~~~~~~][decode][WAIT GCS ~~~~~~~~][decode]
Thread 2: [WAIT GCS ~~~~~~~~][decode][WAIT GCS ~~~~~~~~][decode]
Thread 3: [WAIT GCS ~~~~~~~~][decode][WAIT GCS ~~~~~~~~][decode]
Thread 4: [WAIT GCS ~~~~~~~~][decode][WAIT GCS ~~~~~~~~][decode]

CPU Cores: [idle~~~][busy][idle~~~~~~~~~~~][busy][idle~~~]
```

```
Average CPU utilization: ~15-25%  ← CPUs mostly idle!
```

With threads = CPUs × 4 (optimal):

```
Threads 1-4:   [WAIT][decode][WAIT][decode]...
Threads 5-8:   [WAIT][decode][WAIT][decode]...
Threads 9-12:  [WAIT][decode][WAIT][decode]...
Threads 13-16: [WAIT][decode][WAIT][decode]...

CPU Cores: Always have work from threads finishing I/O

Average CPU utilization: ~60-80%  ← Much better!
```

Recommended Thread Configuration

Pod vCPUs	`max_num_threads`	Rationale
1	8	8x multiplier for I/O bound
2	12-16	Good parallelism
4	16-24	Matches KuzuDB benchmarks
8	24-32	Diminishing returns beyond this

Upper bounds:

- GCS rate limits (~5,000 reads/sec per bucket)
- Memory per thread (~10-50MB)
- Kernel scheduling overhead

Buffer Pool Configuration

What Buffer Pool Does

Phase	Buffer Pool Usage
COPY FROM	Caches pages being written, handles overflow
Idle	Minimal usage
Queries	Caches frequently accessed data pages
Algorithms	**Not used** - NetworkX uses Python heap

Sizing Guidelines

Formula:

```python
def optimal_buffer_pool(pod_memory_limit_bytes: int) -> int:
    """
    Calculate optimal buffer pool size.

    Guidelines:
    - Min 512MB (KuzuDB minimum effective)
    - Max 2GB (diminishing returns beyond this for typical workloads)
    - ~25% of pod limit (leave room for algorithms)
    """
    min_buffer = 512 * 1024 * 1024       # 512MB
    max_buffer = 2 * 1024 * 1024 * 1024  # 2GB
    target_ratio = 0.25                   # 25% of limit

    calculated = int(pod_memory_limit_bytes * target_ratio)
    return max(min_buffer, min(calculated, max_buffer))
```

Examples:

Pod Memory Limit	Buffer Pool	Rationale
4 GB	1 GB	Leave 3GB for Python/algorithms
6 GB	1.5 GB	Balanced
8 GB	2 GB	Capped at 2GB
16 GB	2 GB	Diminishing returns beyond 2GB

Why Not Larger?

From benchmarks:

- Buffer pool has **diminishing returns** for COPY FROM (I/O bound)
- Query caching benefits plateau around 2GB for typical graph sizes
- Larger buffer pool = less memory for NetworkX algorithms
- Disk spilling handles overflow efficiently

Write Serialization Impact

What Can and Cannot Be Parallelized

CAN be parallelized (within single COPY FROM):

- Multiple READER threads fetch files concurrently
- Parquet decoding happens in parallel
- GCS requests are concurrent

CANNOT be parallelized (between COPY FROM statements):

- Each COPY FROM is a write transaction
- KuzuDB serializes write transactions
- COPY Customer → COPY Product → COPY PURCHASED must be sequential

Correct Loading Pattern

```
# Sequential approach (correct)
for node_def in node_definitions:
    # Internal parallelism: multiple threads read Parquet files
    conn.execute(f"COPY {node_def.label} FROM 'gs://.../*.parquet'")

for edge_def in edge_definitions:
    # Internal parallelism: multiple threads read Parquet files
    conn.execute(f"COPY {edge_def.type} FROM 'gs://.../*.parquet'")
```

Application-level parallelism does NOT help because:

1. Single connection is not thread-safe
2. Multiple connections still serialize writes internally
3. The only parallelism is WITHIN each COPY FROM

Kubernetes Memory Configuration

Pod Memory Phases

```
Memory Usage Over Pod Lifecycle

  8GB ┤                                ┌──────┐ Algorithm
      │                                │      │ Execution
  6GB ┤                          ┌─────┘      └────┐
      │          ┌──────────┐    │                 │
  4GB ┤    ┌─────┘          └────┘                 └────┐
      │    │ COPY FROM                                  │
  2GB ┤────┘                                            └────
      │ Startup              Idle      Query    Idle
  0GB ┼──────────────────────────────────────────────────────
      Time →
```

Phase	Memory Profile	Duration
Startup	~500MB (Python, libs)	10-30s
COPY FROM	1-3GB (buffer pool active)	30s-5min
Idle/Queries	1-2GB (buffer pool + overhead)	Majority
Algorithm	3-8GB **spike** (NetworkX in heap)	10s-10min

QoS Classes

From Kubernetes Pod QoS:

QoS Class	Condition	Eviction Priority
Guaranteed	requests == limits	Last (highest priority)

QoS Class	Condition	Eviction Priority
Burstable	requests < limits	Middle
BestEffort	No requests/limits	First (lowest priority)

Configuration Strategies

Strategy A: Guaranteed (Production-Critical)

```yaml
resources:
  requests:
    memory: "4Gi"
    cpu: "1000m"
  limits:
    memory: "4Gi"    # Same = Guaranteed QoS
    cpu: "2000m"

env:
  - name: BUFFER_POOL_SIZE
    value: "1610612736"  # 1.5GB
  - name: MAX_THREADS
    value: "8"
```

Aspect	Value
QoS	Guaranteed
Pods per 32GB node	~7
OOM Risk	Low
Cost	Higher

Strategy B: Smart Burstable (Balanced)

```yaml
resources:
  requests:
    memory: "2Gi"
    cpu: "500m"
  limits:
    memory: "6Gi"
    cpu: "2000m"
```

```yaml
env:
  - name: BUFFER_POOL_SIZE
    value: "1073741824"  # 1GB
  - name: MAX_THREADS
    value: "16"
```

Aspect	Value
QoS	Burstable
Pods per 32GB node	~12-14
OOM Risk	Medium
Cost	Moderate

Strategy C: Dedicated Node Pool (Recommended)

```yaml
resources:
  requests:
    memory: "3Gi"
    cpu: "1000m"
  limits:
    memory: "8Gi"
    cpu: "2000m"

env:
  - name: BUFFER_POOL_SIZE
    value: "2147483648"  # 2GB
  - name: MAX_THREADS
    value: "16"
```

With dedicated node pool (n2-highmem-4, 32GB):

Aspect	Value
QoS	Burstable (isolated)
Pods per node	3-4
OOM Risk	Very Low

Aspect	Value
Cost	Optimal

GKE Node Pool Configuration

Recommended Machine Type

Machine	vCPU	Memory	Memory/CPU	Recommendation
e2-standard-4	4	16 GB	4 GB/vCPU	Budget
e2-highmem-4	4	32 GB	8 GB/vCPU	Good
n2-highmem-4	4	32 GB	8 GB/vCPU	**Recommended**
n2-highmem-8	8	64 GB	8 GB/vCPU	Large clusters

n2-highmem-4 provides the best balance of memory capacity and cost for graph workloads.

Dedicated Node Pool Setup

```
# Terraform example
resource "google_container_node_pool" "graph_instances" {
  name    = "graph-instances"
  cluster = google_container_cluster.primary.name

  node_config {
    machine_type = "n2-highmem-4"

    taint {
      key    = "workload"
      value  = "graph-instance"
      effect = "NO_SCHEDULE"
    }

    labels = {
      "workload-type" = "graph-instance"
    }
  }
```

```
  autoscaling {
    min_node_count = 0
    max_node_count = 10
  }
}
```

Pod Tolerations

```yaml
spec:
  tolerations:
    - key: "workload"
      operator: "Equal"
      value: "graph-instance"
      effect: "NoSchedule"
  nodeSelector:
    workload-type: graph-instance
```

Recommended Configuration Summary

Environment Variables

```yaml
env:
  - name: BUFFER_POOL_SIZE
    value: "2147483648"    # 2GB
  - name: MAX_THREADS
    value: "16"            # 4x vCPU for I/O-bound GCS
```

Pod Resources

```yaml
resources:
  requests:
    memory: "3Gi"
    cpu: "1000m"
  limits:
    memory: "8Gi"
    cpu: "2000m"
```

Memory Budget

```
┌──────────────────────────────────────────┐
│        8 GB Pod Memory Limit             │
├──────────────────────────────────────────┤
│  Python + FastAPI + libs       ~500 MB   │
│  KuzuDB overhead               ~200 MB   │
│  Buffer pool                  2,048 MB   │
│  ──────────────────────────────────────  │
│  Available for algorithms     ~5,250 MB  │
└──────────────────────────────────────────┘
```

Node Pool

- Machine type: n2-highmem-4 (32GB RAM, 4 vCPU)
- Pods per node: 3-4 (safe burst capacity)
- Autoscaling: 0-10 nodes
- Taints: Isolate graph workloads

Performance Comparison

Configuration	COPY FROM	Queries	Algorithms	Stability
Current (512Mi/4Gi, 2GB buffer)	Good	Good	**Risk OOM**	Poor
Strategy A (4Gi/4Gi, 1.5GB)	Good	Good	Limited	Excellent
Strategy B (2Gi/6Gi, 1GB)	Good	Moderate	Good	Good
Strategy C (3Gi/8Gi, 2GB)	**Excellent**	**Good**	**Excellent**	**Excellent**

References

KuzuDB Documentation

- KuzuDB COPY Pipeline Architecture
- KuzuDB 0.7.0 Release - Benchmarks
- The Data Quarry - Kuzu Performance Analysis

GCP/Kubernetes Documentation

- Kubernetes Pod QoS Classes
- GKE Pod Bursting
- GKE Node Sizing

Related Issues

- DuckDB GCS Performance Issue - I/O patterns for cloud storage